

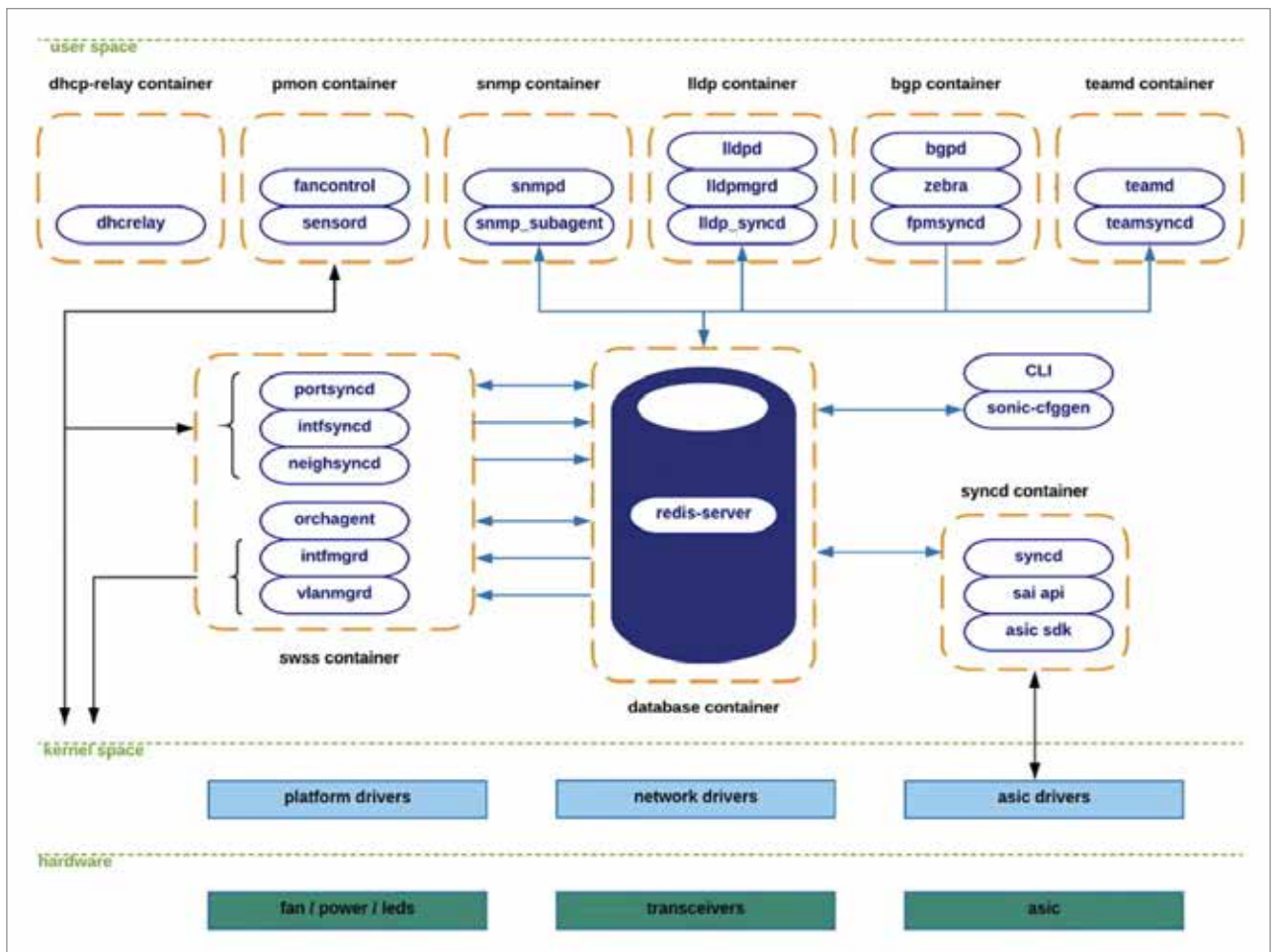


Figure 1: SONiC layers and SAI interface (Courtesy: <https://sonicfoundation.dev/deep-dive-into-sonic-architecture-design/>)

container. This enables components to be upgraded or restarted in isolation, significantly improving the system's overall reliability and allowing for faster innovation and greater hardware flexibility.

When you walk into a well-run restaurant, you rarely think about what happens behind the scenes. Orders move smoothly from the table to the kitchen, chefs coordinate without shouting, and dishes arrive hot and on time.

Modern data centre switches aim for the same experience. SONiC—the open source network operating system used by hyperscalers—was designed so that dozens of independent software components can cooperate quietly in the background. To make that architecture easier to grasp, let's borrow the

restaurant analogy and see how SONiC keeps packets flowing as reliably as meals in a busy dining hall.

How orders enter the system

Every restaurant begins with customers placing requests. In SONiC, those 'orders' come from operators and automation systems through the command-line interface, configuration files, or management APIs.

These inputs describe what the switch should do—add a route, create a VLAN, enable telemetry—without dictating how the hardware must implement them.

The head waiter of SONiC

In a restaurant, a head waiter collects orders and ensures they reach the right

kitchen station. SONiC assigns that role to its orchestration agents, often referred to collectively as OrchAgent. They read the intended state from a shared database and translate it into concrete tasks for lower layers.

This separation allows high-level intent to remain stable even if hardware platforms or vendor drivers change underneath.

What happens in the kitchen

Behind the swinging doors sits the kitchen manager. In SONiC, that job belongs to the process that talks to the switch silicon through the Switch Abstraction Interface, or SAI.

SAI acts like a standard recipe book. It defines how to program forwarding tables, access-control lists, counters, and